

Q-1 What is a cursor? Why do we need Cursor?

Ans: A cursor is a pointer that points to the result set of an SQL query.
It is used to process records row-wise or one row at a time

Why do we need a cursor?

First, the basic problem

SQL alone:

```
SELECT * FROM student;
```

This can return many rows.

A cursor allows PL/SQL to:

- Hold multiple rows
- Read one row at a time (can do arithmetic operation on that row)
- Process each record step by step (store data in a variable and process it)

PL/SQL variables can store only ONE row at a time.

So what is the solution?

☞ Cursor

◇ Simple Definition

A cursor is used in PL/SQL to fetch and process multiple rows returned by a SELECT statement one row at a time.

◇ Real-Life Example

Imagine:

You have a list of students written on paper.

You:

Read one name

Do something

Go to the next name

☞ That list is the cursor

☞ Reading one student at a time is FETCH

◇ Without Cursor vs With Cursor

✗ Without Cursor (Not possible)

```
SELECT name INTO v_name FROM student;
```

Fails if table has many rows.

☒ With Cursor (Correct)

```
DECLARE
```

```
    CURSOR c1 IS SELECT name FROM student;
```

```
BEGIN
```

```
    FOR r IN c1 LOOP
```

```
        DBMS_OUTPUT.PUT_LINE(r.name);
```

```
    END LOOP;
```

```
END;
```

```
/
```

✓ Reads one student at a time

✓ No error

✓ Can process all rows

◇ When exactly do we NEED a cursor?

We need a cursor when:

✓ SELECT returns more than one row

✓ We want to apply logic to each row

✓ We want to update rows one by one

✓ We want to display records row-wise

◇ Simple Comparison (Very Important)

Situation	Cursor Needed?
INSERT	✗ No
UPDATE	✗ No
DELETE	✗ No
SELECT INTO (1 row)	✗ No
SELECT (many rows)	☑ YES

Thus, Cursor is needed in PL/SQL to handle and process multiple rows returned by a SELECT statement, as PL/SQL variables can store only one row at a time.

A cursor is a pointer that allows PL/SQL to fetch and process multiple rows returned by a SELECT query one row at a time. It is required because PL/SQL does not support storing multiple rows in a single variable.

SQL works on many rows together, but PL/SQL works on one row at a time.

Types of Cursor:

1. Implicit Cursor
2. Explicit Cursor

IMPLICIT CURSOR:

Automatically created by Oracle for INSERT, UPDATE, DELETE, SELECT INTO.

Implicit Cursor Attributes:

SQL%FOUND – TRUE if row affected

SQL%NOTFOUND – TRUE if no row affected

SQL%ROWCOUNT – Number of rows affected

SQL%ISOPEN – Always FALSE

EXPLICIT CURSOR:

Created by programmer to handle multiple rows.

Steps: DECLARE → OPEN → FETCH → CLOSE

(For each syntax refer material unit-1)

Example Tables:

STUDENT(stud_id, name, marks)

EMP(emp_id, name, salary, dept)

IMPLICIT CURSOR – SOLVED PROGRAMS

1.--implicit cursor isopen and rowcount attribute

set serveroutput on;

BEGIN

 UPDATE stud

 SET marks = marks + 2;

 IF (SQL%ISOPEN) THEN

 DBMS_OUTPUT.PUT_LINE('Cursor is open');

 ELSE

 DBMS_OUTPUT.PUT_LINE('cursor is closed Rows affected: ' || SQL%ROWCOUNT);

 END IF;

END;

/

2.-- implicit %found attribute

begin

 update stud set marks=100 where marks>90;

 if (sql%found) then

 dbms_output.put_line('data updated');

 else

 dbms_output.put_line('data not found');

 end if;

end; /

3.-- Row count

BEGIN

UPDATE stu

SET class = '10B'

WHERE age = 15;

DBMS_OUTPUT.PUT_LINE('Rows affected: ' || SQL%ROWCOUNT);

END;

/

4.-- found

BEGIN

DELETE FROM stu

WHERE gender = 'Male';

IF SQL%FOUND THEN

DBMS_OUTPUT.PUT_LINE('Male student records deleted');

END IF;

END;

/

5. Update student marks

BEGIN

UPDATE student SET marks=85 WHERE stud_id=1;

IF SQL%FOUND THEN

DBMS_OUTPUT.PUT_LINE('Updated');

END IF;

END;

/

6. Delete student

BEGIN

```
DELETE FROM student WHERE stud_id=1;
DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT);
END;
/
```

7. Check record

```
BEGIN
UPDATE student SET marks=90 WHERE stud_id=5;
IF SQL%NOTFOUND THEN

    DBMS_OUTPUT.PUT_LINE('No Record');

END IF;
END;
/
```

8. Delete failed students

```
BEGIN
DELETE FROM student WHERE marks<40;
DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT);
END;
/
```

EXPLICIT CURSOR solved programs

1. --explicit cursor

```
DECLARE

CURSOR emp_cur IS

    SELECT emp_name, salary FROM emp;

v_name  emp.emp_name%TYPE;
v_salary emp.salary%TYPE;

BEGIN

OPEN emp_cur;

LOOP

    FETCH emp_cur INTO v_name, v_salary;

    EXIT WHEN emp_cur%NOTFOUND;
```

```
    DBMS_OUTPUT.PUT_LINE(v_name || ' earns ' || v_salary);  
END LOOP;  
CLOSE emp_cur;  
END;  
/
```

2.--parameterised cursor

```
DECLARE  
CURSOR c_emp (p_deptno emp.deptno%TYPE) IS  
SELECT emp_id, emp_name, salary  
FROM emp  
WHERE deptno = p_deptno;  
BEGIN  
FOR r IN c_emp(10) LOOP  
DBMS_OUTPUT.PUT_LINE(r.emp_id || ' ' || r.emp_name || ' ' || r.salary);  
END LOOP;  
END;  
/
```

3.--cursor for loop

```
declare  
cursor cur_emp is select emp_name,salary from emp;  
v_total number(7) :=0;  
begin  
for r in cur_emp  
loop  
dbms_output.put_line(r.emp_name || ' earns ' || r.salary);
```

```
v_total:=V_total+r.salary;

end loop;

dbms_output.put_line('total salary is :- ' || v_total);

end;

/
```

4. --Display all students

```
DECLARE
CURSOR c1 IS SELECT * FROM student;
BEGIN
FOR r IN c1 LOOP
DBMS_OUTPUT.PUT_LINE(r.stud_id || ' ' || r.name || ' ' || r.marks);
END LOOP;
END;

/
```

5. --Students >70

```
DECLARE
CURSOR c1 IS SELECT name,marks FROM student WHERE marks>70;
BEGIN
FOR r IN c1 LOOP
DBMS_OUTPUT.PUT_LINE(r.name || ' ' || r.marks);
END LOOP;
END;

/
```

6. --Display employees

```
DECLARE
CURSOR c1 IS SELECT emp_id,name,salary FROM emp;
BEGIN
FOR r IN c1 LOOP
DBMS_OUTPUT.PUT_LINE(r.emp_id || ' ' || r.name || ' ' || r.salary);
END LOOP;
END;

/
```

7.-- Count employees

```
DECLARE
```



```
CURSOR c1 IS SELECT * FROM emp;
cnt NUMBER:=0;
BEGIN
FOR r IN c1 LOOP cnt:=cnt+1; END LOOP;
DBMS_OUTPUT.PUT_LINE(cnt);
END;
/
```

8.-- IT employees

```
DECLARE
CURSOR c1 IS SELECT name FROM emp WHERE dept='IT';
BEGIN
FOR r IN c1 LOOP DBMS_OUTPUT.PUT_LINE(r.name); END LOOP;
END;
/
```

9.-- Increase salary using cursor

```
DECLARE
CURSOR c1 IS SELECT emp_id FROM emp;
BEGIN
FOR r IN c1 LOOP
UPDATE emp SET salary=salary+1000 WHERE emp_id=r.emp_id;
END LOOP;
END;
/
```

10.-- Student grades

```
DECLARE
CURSOR c1 IS SELECT name,marks FROM student;
BEGIN
FOR r IN c1 LOOP
IF r.marks>=75 THEN DBMS_OUTPUT.PUT_LINE(r.name||' Distinction');
ELSE DBMS_OUTPUT.PUT_LINE(r.name||' Pass'); END IF;
END LOOP;
END;
/
```

11.-- Cursor with parameter

```
DECLARE
CURSOR c1(d emp.dept%TYPE) IS SELECT name FROM emp WHERE dept=d;
BEGIN
```

```
FOR r IN c1('IT') LOOP DBMS_OUTPUT.PUT_LINE(r.name); END LOOP;  
END;  
/
```

12.-- Highest salary

```
DECLARE  
CURSOR c1 IS SELECT name,salary FROM emp ORDER BY salary DESC;  
BEGIN  
FOR r IN c1 LOOP  
DBMS_OUTPUT.PUT_LINE(r.name || ' ' || r.salary);  
EXIT;  
END LOOP;  
END;  
/
```

13.-- Cursor rowcount

```
DECLARE  
CURSOR c1 IS SELECT * FROM student;  
BEGIN  
OPEN c1;  
FETCH c1;  
DBMS_OUTPUT.PUT_LINE(c1%ROWCOUNT);  
CLOSE c1;  
END;  
/
```